Android Testing — Cheat Sheet

Unit · Integration · UI · KMP-friendly · 2026 stack with practical examples for every layer

Sample project | JUnit 5 · Kotest · MockK · Turbine | Hilt + Compose + Paparazzi | SDE3-ready

About this sheet: a focused, practical guide to testing Android in 2026. We define a small sample project (a Notes app) and write tests for *every* layer of it — plain class, UseCase, Repository, ViewModel, Room DAO, Retrofit Api, Composable, Service, ContentProvider, DataStore. Same UI shell as the other cheat sheets in the repo: dark mode, search (⌘K), sticky TOC, copy-code, print-ready PDF. All code is original and uses standard idiomatic patterns.

1. Why testing? — the problem and the cost of not testing

Tests aren't optional ceremony. They're the only way to keep a non-trivial codebase changeable without fear. Without tests:

- Every refactor is a roll of the dice — you only find out you broke something when QA or users do
- Onboarding new devs takes weeks because they're scared to touch anything
- Bug fix → ship → new bug emerges in seemingly unrelated code (regression)
- Performance work becomes guesswork instead of data-driven

The cost-of-bug curve

WHERE BUG IS CAUGHT	COST TO FIX
While writing the code (compiler / IDE)	~1×
During unit tests on local machine	~5×
During CI (PR review)	~10×
During QA / staging	~50×
In production (1-star reviews)	~500× + reputation cost

Tests pull bug detection left on this curve. A good test suite turns a 500× incident into a 5× minor commit.

What good tests give you

- **Confidence to refactor** — change implementation freely; tests verify behavior didn't break
- **Living documentation** — tests show how a class is meant to be used (real usage, not stale comments)
- **Design feedback** — hard-to-test code is usually badly designed code

- **Regression prevention** — every fixed bug becomes a test that prevents it from coming back
- **Faster onboarding** — new devs trust the safety net

2. The testing pyramid — types of tests and when to use which

```
                /\
               /UI\          ← E2E / instrumented (slow, fragile, expensive)
              /----\            ~5% of total tests
             / Comp \         ← Compose / Espresso UI tests (mid)
            /--------\           ~15% of tests
           /Integration\       ← Multi-layer (Repo + DAO, MockWebServer + Api)
          /------------\         ~20% of tests
         /     UNIT     \      ← Pure JVM, fast, in millions, no Android
        /----------------\       ~60% of tests
       /                  \
```

LEVEL	SPEED	CONFIDENCE PER TEST	WHAT TO TEST THERE
Unit	ms (JVM only)	One class, one behavior	UseCases, ViewModels, Repositories, plain classes, reducers, validators
Integration	seconds	Multi-layer correctness	Repo + Room (in-memory DB), Api + MockWebServer, ViewModel + real DataStore
UI (Compose / Espresso)	seconds (instrumented)	Renders correctly + interactions work	Single-screen flows; key user gestures; accessibility
E2E (Maestro / UIAutomator)	tens of seconds	Full user journey	Critical flows only — login, checkout, sign-up
Macrobenchmark	seconds (release build)	Performance regressions	Cold start, scrolling, key flows
Screenshot	seconds (JVM via Paparazzi)	Visual regressions	Design system components, key screens in different states/themes

> **Rule of thumb:** push every test as low on the pyramid as it can go. A bug caught at unit level costs 5×; at UI level costs 50×. Same bug, 10× the cost.

3. Testing vocabulary — fakes, mocks, stubs, spies

TERM	WHAT IT MEANS	WHEN USE
System under test (SUT)	The class you're actually testing	Always the focus
Fixture	The setup data / state shared across tests	JUnit's `@BeforeEach`
Test double	Umbrella term for any stand-in (fake, mock, stub, spy, dummy)	Whenever you replace a real dependency
Dummy	Object passed but never used (just to satisfy a parameter)	Rare; mostly when you only care about other args
Stub	Pre-programmed answers — "if asked, return X"	When SUT only reads
Mock	Stub + verifies interactions ("was method X called?")	When SUT's correctness is "did it call this?"
Spy	Real object + records calls (partial mock)	Rare; testing legacy classes you can't refactor
Fake	Working implementation, simplified for tests (e.g., in-memory DB)	**Default choice for repositories and data sources**

Fake vs Mock — the most important decision

PREFER FAKES FOR REPOSITORY-LIKE DEPENDENCIES

- Same interface as real, but uses an in-memory backing
- Tests look like normal code — easy to read
- Survives interface refactors (you only update one fake, not 50 mock setups)
- Catches contract bugs (your fake exhibits same edge cases)

USE MOCKS SPARINGLY — ONLY FOR VERIFICATION

- Mocks tightly couple tests to implementation details
- Refactor breaks 50 mock setups even if behavior is identical
- Use only when "did the SUT call this method?" is the actual property under test (e.g., did Analytics.track fire?)

```kotlin
// FAKE — preferred for Repository
class FakeNotesRepository : NotesRepository {
    private val notes = mutableMapOf<String, Note>()
    private val flow = MutableStateFlow<List<Note>>(emptyList())
    override fun observe(): Flow<List<Note>> = flow
    override suspend fun add(note: Note) { notes[note.id] = note; flow.value = notes.values.toList() }
    override suspend fun get(id: String) = notes[id]
}

// MOCK — only when verifying interaction is the test's purpose
val analytics = mockk<Analytics>(relaxed = true)
vm.onClick()
verify { analytics.track("button_clicked") }      // the test IS about this call
```

4. The latest stack (2026) — what to put in your `build.gradle.kts`

PURPOSE	LIBRARY	NOTES
Test runner / framework	**JUnit 5 (Jupiter)** for JVM tests, `de.mannodermaus.android-junit5` Gradle plugin for Android	Better than JUnit 4: nested tests, parameterized cleaner, dynamic tests, lifecycle hooks more flexible
KMP-friendly framework	**Kotest** or **kotlin.test**	Both work in `commonTest`; Kotest has nicer matchers and BDD-style
Mocking	**MockK**	Kotlin-first; supports final classes by default (Mockito needs mockito-inline); supports suspend, top-level functions, object/companion mocking
Coroutines testing	`kotlinx-coroutines-test`	Provides `runTest`, `StandardTestDispatcher`, virtual time
Flow testing	**Turbine** (cash app)	Cleanest API for asserting Flow emissions one by one
HTTP mocking	**OkHttp MockWebServer**	Real OkHttp client hits a local server you control
JVM-side Android API testing	**Robolectric**	Run Android-API code on JVM (no emulator)
Compose UI tests	`androidx.compose.ui:ui-test-junit4` (or junit5 variant)	`createComposeRule` / `createAndroidComposeRule`
Hilt testing	`com.google.dagger:hilt-android-testing`	`@HiltAndroidTest`, `HiltAndroidRule`, `@TestInstallIn`
Screenshot testing	**Paparazzi** (View) or **Roborazzi** (Compose-friendly)	JVM-only — no emulator needed
Macrobenchmark	`androidx.benchmark:benchmark-macro-junit4`	Measure cold start, scrolling on release builds
End-to-end UI flows	**Maestro** or UI Automator	Maestro is YAML-based, much simpler than Espresso for E2E
Code coverage	**Kover** (Kotlin/KMP) or JaCoCo (JVM-only)	Kover preferred for KMP modules
Truth / assertions	**kotlin.test** (KMP) or **Google Truth**	Avoid Hamcrest in 2026 — verbose
Property-based testing (advanced)	**Kotest Property**	Generate hundreds of random inputs; great for parsers, validators

Skeleton `libs.versions.toml` entries

```
[versions]
junit-jupiter = "5.10.2"
junit5-android = "1.10.0.0"
mockk = "1.13.10"
turbine = "1.1.0"
coroutines = "1.8.1"
mockwebserver = "4.12.0"
robolectric = "4.13"
compose-bom = "2024.06.00"
hilt = "2.51.1"
paparazzi = "1.3.4"
kotest = "5.9.0"

[libraries]
junit-jupiter-api    = { module = "org.junit.jupiter:junit-jupiter-api", version.ref = "junit-jupiter" }
junit-jupiter-engine = { module = "org.junit.jupiter:junit-jupiter-engine", version.ref = "junit-jupiter" }
mockk                = { module = "io.mockk:mockk", version.ref = "mockk" }
turbine              = { module = "app.cash.turbine:turbine", version.ref = "turbine" }
coroutines-test      = { module = "org.jetbrains.kotlinx:kotlinx-coroutines-test", version.ref = "coroutines"
mockwebserver        = { module = "com.squareup.okhttp3:mockwebserver", version.ref = "mockwebserver" }
robolectric          = { module = "org.robolectric:robolectric", version.ref = "robolectric" }
hilt-testing         = { module = "com.google.dagger:hilt-android-testing", version.ref = "hilt" }
hilt-compiler        = { module = "com.google.dagger:hilt-android-compiler", version.ref = "hilt" }
compose-bom          = { module = "androidx.compose:compose-bom", version.ref = "compose-bom" }
compose-test-junit4  = { module = "androidx.compose.ui:ui-test-junit4" }
kotest-runner-jvm    = { module = "io.kotest:kotest-runner-junit5", version.ref = "kotest" }
kotest-assertions    = { module = "io.kotest:kotest-assertions-core", version.ref = "kotest" }
```

Test source-set layout

```
src/
├── main/                    // production code
├── test/                    // JVM-only tests (fast, run anywhere)
│   └── kotlin/...
├── androidTest/             // instrumented tests (need emulator/device)
│   └── kotlin/...
└── commonTest/              // (KMP only) shared tests
    └── kotlin/...
```

5. Designing for testability — the 7 principles

1. **Depend on interfaces, not implementations.** ViewModel takes `UserRepository` (interface), not `UserRepositoryImpl`. Lets you swap a fake.
2. **Inject dependencies through constructor.** No `val api = Retrofit.Builder()...build()` inside classes — receive what you need.
3. **No hidden global state.** Singletons that hold mutable state are test poison. Use Hilt scopes for true singletons; never `object Manager { var x }`.

4. **Inject the dispatcher.** Don't hardcode `Dispatchers.IO`; receive it. Tests inject `StandardTestDispatcher` for virtual time.
5. **Inject the clock.** Don't call `System.currentTimeMillis()` directly — wrap in a `Clock` interface. Tests inject a fixed clock for time-dependent logic.
6. **Pure functions wherever possible.** Reducers, validators, formatters — all should be pure. Easiest tests in the world.
7. **Avoid static methods, top-level functions with side effects, and singletons that touch the OS.** Wrap them in classes that can be replaced.

```
// Bad — untestable
class Repo {
    private val api = Retrofit.Builder()...build().create<Api>()    // hidden dep
    suspend fun get() = withContext(Dispatchers.IO) { api.fetch() }    // hidden dispatcher
    fun isExpired(token: Token) = token.expiresAt < System.currentTimeMillis()  // hidden clock
}

// Good — testable
class Repo @Inject constructor(
    private val api: Api,                       // inject interface
    private val ioDispatcher: CoroutineDispatcher, // inject dispatcher
    private val clock: Clock                    // inject clock
) {
    suspend fun get() = withContext(ioDispatcher) { api.fetch() }
    fun isExpired(token: Token) = token.expiresAt < clock.now()
}
```

6. The sample project — a Notes app we'll test layer by layer

For the rest of this guide we'll test a small Notes app. Here are the contracts (interfaces); implementations are obvious. Every later section tests one of these layers in isolation.

```
// Domain model
data class Note(val id: String, val title: String, val body: String, val updatedAt: Long)

// Repository interface
interface NotesRepository {
    fun observeAll(): Flow<List<Note>>
    suspend fun get(id: String): Note?
    suspend fun save(note: Note)
    suspend fun delete(id: String)
    suspend fun sync()
}

// UseCase
class SaveNoteUseCase @Inject constructor(
    private val repo: NotesRepository,
    private val clock: Clock
) {
    suspend operator fun invoke(title: String, body: String): Result<Note> {
        if (title.isBlank()) return Result.failure(IllegalArgumentException("empty title"))
        val note = Note(id = UUID.randomUUID().toString(), title = title, body = body, updatedAt = clock.now()
        repo.save(note)
        return Result.success(note)
    }
}

// ViewModel
@HiltViewModel
class NotesViewModel @Inject constructor(
    private val repo: NotesRepository,
    private val save: SaveNoteUseCase
) : ViewModel() {
    val notes: StateFlow<List<Note>> = repo.observeAll()
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), emptyList())

    private val _events = Channel<UiEvent>(Channel.BUFFERED)
    val events: Flow<UiEvent> = _events.receiveAsFlow()

    fun add(title: String, body: String) = viewModelScope.launch {
        save(title, body)
            .onSuccess { _events.send(UiEvent.Saved) }
            .onFailure { _events.send(UiEvent.Error(it.message ?: "unknown")) }
    }

    sealed interface UiEvent {
        data object Saved : UiEvent
        data class Error(val msg: String) : UiEvent
    }
}

// Composable
@Composable
```

```
fun NotesScreen(vm: NotesViewModel = hiltViewModel()) {
    val notes by vm.notes.collectAsStateWithLifecycle()
    Column {
        notes.forEach { Text(it.title, Modifier.testTag("note_${it.id}")) }
        Button(onClick = { vm.add("New", "Body") },
               modifier = Modifier.testTag("add_btn")) { Text("Add") }
    }
}
```

7. Test: plain Kotlin class

The simplest possible test. Pure logic, no dependencies, no Android. Use this every time you have a function that should be a pure function — extracted from somewhere bigger.

```
// Production class
class EmailValidator {
    fun isValid(email: String): Boolean =
        email.matches(Regex("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$"))
}

// JUnit 5 test
class EmailValidatorTest {
    private val validator = EmailValidator()

    @Test
    fun `accepts standard email`() {
        assertTrue(validator.isValid("sai@example.com"))
    }

    @Test
    fun `rejects email without @`() {
        assertFalse(validator.isValid("saiexample.com"))
    }

    @ParameterizedTest
    @ValueSource(strings = ["", "a@b", "@example.com", "a@.com"])
    fun `rejects invalid emails`(email: String) {
        assertFalse(validator.isValid(email))
    }
}
```

Best practices:

- Use backticked function names — they read like sentences in test reports
- One behavior per test (Arrange / Act / Assert pattern)
- Use parameterized tests for "same logic, multiple inputs" — keeps test count up while code stays compact
- Don't test private methods; test through public methods. If a private method is too complex to test that way, extract it into its own class with public methods

8. Test: UseCase (pure logic with injected deps)

UseCases are mostly orchestration: validate input, call the repo, format output. Inject everything; assert on Result.

```
class SaveNoteUseCaseTest {
    private val fakeRepo = FakeNotesRepository()
    private val fixedClock = object : Clock { override fun now() = 1_700_000_000_000L }
    private val useCase = SaveNoteUseCase(fakeRepo, fixedClock)

    @Test
    fun `saves note when title valid`() = runTest {
        val result = useCase("Title", "Body")
        assertTrue(result.isSuccess)
        val note = result.getOrThrow()
        assertEquals("Title", note.title)
        assertEquals(1_700_000_000_000L, note.updatedAt)
        assertEquals(1, fakeRepo.allSnapshot().size)
    }

    @Test
    fun `fails when title blank`() = runTest {
        val result = useCase("", "body")
        assertTrue(result.isFailure)
        assertTrue(result.exceptionOrNull() is IllegalArgumentException)
        assertEquals(0, fakeRepo.allSnapshot().size)
    }
}

// FakeNotesRepository
class FakeNotesRepository : NotesRepository {
    private val store = mutableMapOf<String, Note>()
    private val flow = MutableStateFlow<List<Note>>(emptyList())

    override fun observeAll(): Flow<List<Note>> = flow
    override suspend fun get(id: String) = store[id]
    override suspend fun save(note: Note) { store[note.id] = note; flow.value = store.values.toList() }
    override suspend fun delete(id: String) { store.remove(id); flow.value = store.values.toList() }
    override suspend fun sync() { /* no-op for tests */ }

    // Test-only helpers
    fun allSnapshot(): List<Note> = store.values.toList()
    fun seed(notes: List<Note>) { notes.forEach { store[it.id] = it }; flow.value = store.values.toList() }
}
```

9. Test: Repository (with fake DAO + fake Api)

Repositories orchestrate data sources. Test "given network returns X, repo emits Y" — without spinning up Room or OkHttp.

```
class NotesRepositoryImplTest {
    private val fakeDao = FakeNotesDao()
    private val fakeApi = FakeNotesApi()
    private val repo = NotesRepositoryImpl(fakeDao, fakeApi)

    @Test
    fun `sync writes remote results to DAO`() = runTest {
        fakeApi.notesToReturn = listOf(Note("1", "a", "", 0), Note("2", "b", "", 0))
        repo.sync()
        assertEquals(2, fakeDao.allSnapshot().size)
    }

    @Test
    fun `observeAll emits from DAO`() = runTest {
        fakeDao.seed(listOf(Note("x", "x", "", 0)))
        repo.observeAll().test {
            assertEquals(1, awaitItem().size)
            cancelAndIgnoreRemainingEvents()
        }
    }

    @Test
    fun `save persists locally and queues remote sync`() = runTest {
        val note = Note("1", "t", "b", 0)
        repo.save(note)
        assertTrue(fakeDao.allSnapshot().contains(note))
        assertTrue(fakeApi.savedNotes.contains(note))
    }
}

class FakeNotesDao : NotesDao {
    private val store = mutableMapOf<String, Note>()
    private val flow = MutableStateFlow<List<Note>>(emptyList())
    override fun observeAll() = flow
    override suspend fun upsert(note: Note) { store[note.id] = note; flow.value = store.values.toList() }
    override suspend fun get(id: String) = store[id]
    override suspend fun delete(id: String) { store.remove(id); flow.value = store.values.toList() }
    fun allSnapshot() = store.values.toList()
    fun seed(notes: List<Note>) { notes.forEach { store[it.id] = it }; flow.value = store.values.toList() }
}

class FakeNotesApi : NotesApi {
    var notesToReturn: List<Note> = emptyList()
    val savedNotes = mutableListOf<Note>()
    override suspend fun fetchAll() = notesToReturn
    override suspend fun push(note: Note) { savedNotes += note }
}
```

10. Test: ViewModel (Coroutines + StateFlow + SharedFlow events)

The most-tested layer in a typical project. Three things to verify: state evolves correctly, events fire correctly, no leaks.

```kotlin
@OptIn(ExperimentalCoroutinesApi::class)
class NotesViewModelTest {

    private val dispatcher = StandardTestDispatcher()
    private val fakeRepo = FakeNotesRepository()
    private val useCase = SaveNoteUseCase(fakeRepo, FixedClock(100L))
    private lateinit var vm: NotesViewModel

    @BeforeEach
    fun setUp() {
        Dispatchers.setMain(dispatcher)        // viewModelScope uses Main
        vm = NotesViewModel(fakeRepo, useCase)
    }

    @AfterEach
    fun tearDown() {
        Dispatchers.resetMain()
    }

    @Test
    fun `notes state mirrors repository`() = runTest(dispatcher) {
        vm.notes.test {
            assertEquals(emptyList<Note>(), awaitItem())               // initial
            fakeRepo.seed(listOf(Note("1", "a", "", 0)))
            assertEquals(1, awaitItem().size)
            cancelAndIgnoreRemainingEvents()
        }
    }

    @Test
    fun `add success emits Saved event and persists`() = runTest(dispatcher) {
        vm.events.test {
            vm.add("Title", "Body")
            advanceUntilIdle()                                          // drain pending coroutines
            assertEquals(NotesViewModel.UiEvent.Saved, awaitItem())
        }
        assertEquals(1, fakeRepo.allSnapshot().size)
    }

    @Test
    fun `add empty title emits Error event`() = runTest(dispatcher) {
        vm.events.test {
            vm.add("", "body")
            advanceUntilIdle()
            val evt = awaitItem()
            assertTrue(evt is NotesViewModel.UiEvent.Error)
        }
        assertEquals(0, fakeRepo.allSnapshot().size)
    }
}
```

Three patterns to memorize:

- `Dispatchers.setMain(dispatcher)` in setUp + `resetMain()` in tearDown — required because `viewModelScope` uses `Dispatchers.Main` which is unavailable in JVM tests by default.
- `runTest(dispatcher)` + `advanceUntilIdle()` — drives coroutines deterministically.
- `flow.test { awaitItem() }` — Turbine; the cleanest way to assert Flow emissions.

11. Test: Room DAO (in-memory database, fast)

Room DAO tests need a real Room — the generated implementations are non-trivial. Use an **in-memory database** in `androidTest/`. With Robolectric you can also run Room on the JVM for some scenarios, but instrumented in-memory is the canonical approach.

```
@RunWith(AndroidJUnit4::class)
class NotesDaoTest {
    private lateinit var db: AppDatabase
    private lateinit var dao: NotesDao

    @Before
    fun setUp() {
        val ctx = ApplicationProvider.getApplicationContext<Context>()
        db = Room.inMemoryDatabaseBuilder(ctx, AppDatabase::class.java)
            .allowMainThreadQueries()
            .build()
        dao = db.notesDao()
    }

    @After fun tearDown() = db.close()

    @Test fun insertAndGet() = runTest {
        val note = Note("1", "t", "b", 100)
        dao.upsert(note)
        assertEquals(note, dao.get("1"))
    }

    @Test fun observeAllEmitsOnInsert() = runTest {
        dao.observeAll().test {
            assertEquals(emptyList<Note>(), awaitItem())
            dao.upsert(Note("1", "t", "", 0))
            assertEquals(1, awaitItem().size)
            cancelAndIgnoreRemainingEvents()
        }
    }
}
```

Migration tests use `MigrationTestHelper` — critical because broken migrations crash users on update.

```
class MigrationTest {
    @get:Rule val helper = MigrationTestHelper(
        InstrumentationRegistry.getInstrumentation(),
        AppDatabase::class.java
    )

    @Test fun migrate1to2() {
        val db = helper.createDatabase("test.db", 1).apply {
            execSQL("INSERT INTO Note VALUES('x', 't', 'b', 100)")
            close()
        }
        helper.runMigrationsAndValidate("test.db", 2, true, MIGRATION_1_2)
        // Open with v2 schema and assert data still present + new column populated
    }
}
```

12. Test: Retrofit Api (MockWebServer)

Don't mock Retrofit. Use a real OkHttp client pointed at a local `MockWebServer` — same code path as production.

```
class NotesApiTest {
    private lateinit var server: MockWebServer
    private lateinit var api: NotesApi

    @BeforeEach fun setUp() {
        server = MockWebServer().apply { start() }
        api = Retrofit.Builder()
            .baseUrl(server.url("/"))
            .addConverterFactory(MoshiConverterFactory.create())
            .build()
            .create(NotesApi::class.java)
    }

    @AfterEach fun tearDown() = server.shutdown()

    @Test fun `fetchAll parses successful response`() = runTest {
        server.enqueue(MockResponse().setBody("""
            [{"id":"1","title":"Hello","body":"World","updatedAt":100}]
        """.trimIndent()))

        val notes = api.fetchAll()
        assertEquals(1, notes.size)
        assertEquals("Hello", notes.first().title)

        val recorded = server.takeRequest()
        assertEquals("GET", recorded.method)
        assertEquals("/notes", recorded.path)
    }

    @Test fun `fetchAll throws HttpException on 500`() = runTest {
        server.enqueue(MockResponse().setResponseCode(500))
        assertThrows<HttpException> { api.fetchAll() }
    }
}
```

13. Test: Composable (createComposeRule)

Compose UI tests are fast (no view inflation), instrumented, and very expressive. Use `testTag` for stable selectors.

```
@RunWith(AndroidJUnit4::class)
class NotesScreenTest {

    @get:Rule val compose = createComposeRule()

    @Test fun `clicking add invokes callback`() {
        var addCalled = false
        compose.setContent {
            NotesScreen(
                state = NotesScreenState(notes = emptyList()),
                onAdd = { addCalled = true }
            )
        }
        compose.onNodeWithTag("add_btn").performClick()
        assertTrue(addCalled)
    }

    @Test fun `renders all notes`() {
        compose.setContent {
            NotesScreen(
                state = NotesScreenState(notes = listOf(
                    Note("1", "First", "", 0),
                    Note("2", "Second", "", 0)
                )),
                onAdd = {}
            )
        }
        compose.onNodeWithText("First").assertIsDisplayed()
        compose.onNodeWithText("Second").assertIsDisplayed()
        compose.onAllNodesWithTag("note_", useUnmergedTree = true).assertCountEquals(2)
    }
}
```

Pattern: hoist state out of the Composable so tests pass static state directly. The above `NotesScreen(state, onAdd)` is a stateless variant; the full `NotesScreen()` with ViewModel calls into this stateless one. This separation makes UI tests trivial.

Key APIs:

- `onNodeWithText("X")` / `onNodeWithTag("tag")` / `onNodeWithContentDescription("desc")`
- `performClick()` / `performTextInput("...")` / `performScrollTo()`
- `assertIsDisplayed()` / `assertIsEnabled()` / `assertTextEquals("...")`
- `compose.waitForIdle()` — wait for animations / async work
- `compose.mainClock.autoAdvance = false; advanceTimeBy(ms)` — control time for animation tests

14. Test: Service / ForegroundService

Services have a real lifecycle. Two approaches:

APPROACH	WHEN USE
Refactor logic out of the Service into a plain class	**Default approach.** The Service becomes a thin shell; logic gets unit-tested separately.
Robolectric `ServiceController`	JVM tests for Service lifecycle on Android API surface
Instrumented tests with `ServiceTestRule`	Bound services or when you need real Android runtime

```
// Production — service is thin, business logic extracted
class SyncService : Service() {
    @Inject lateinit var syncEngine: SyncEngine        // the testable thing
    override fun onStartCommand(i: Intent?, flags: Int, id: Int): Int {
        scope.launch { syncEngine.sync() }
        return START_STICKY
    }
}

// Unit-test SyncEngine — no Service involved
class SyncEngineTest {
    @Test fun sync_uploads_pending_items() = runTest {
        val engine = SyncEngine(fakeRepo, fakeApi)
        fakeRepo.queueLocalChanges(listOf(...))
        engine.sync()
        assertTrue(fakeApi.uploadedItems.size == 3)
    }
}
```

For a ForegroundService notification check, prefer a Robolectric test:

```
@RunWith(RobolectricTestRunner::class)
class SyncServiceTest {
    @Test fun `posts foreground notification on start`() {
        val service = Robolectric.buildService(SyncService::class.java)
            .create()
            .startCommand(0, 0)
            .get()
        val notif = ShadowNotificationManager().activeNotifications.firstOrNull()
        assertNotNull(notif)
    }
}
```

15. Test: ContentProvider

Use the `ProviderTestRule` in instrumented tests. It sets up an isolated MockContentResolver pointing at your provider — no real provider registration needed.

```
@RunWith(AndroidJUnit4::class)
class NotesProviderTest {
    @get:Rule val providerRule = ProviderTestRule.Builder(
        NotesProvider::class.java,
        "com.example.notes.provider"
    ).build()

    @Test fun `insert and query round-trip`() {
        val resolver = providerRule.resolver
        val uri = Uri.parse("content://com.example.notes.provider/notes")
        val values = ContentValues().apply {
            put("id", "1"); put("title", "t"); put("body", "b")
        }
        val inserted = resolver.insert(uri, values)
        assertNotNull(inserted)

        resolver.query(uri, null, null, null, null)?.use { c ->
            assertTrue(c.moveToFirst())
            assertEquals("t", c.getString(c.getColumnIndexOrThrow("title")))
        }
    }
}
```

Better pattern: wrap your ContentProvider behind a Repository / DataSource interface. Then your application code talks to the interface, and you can test with a fake — same pattern as Retrofit / Room.

16. Test: SharedPreferences / DataStore

For new code, use DataStore (preferred). Both can be tested cleanly.

SharedPreferences (legacy)

```
@RunWith(RobolectricTestRunner::class)
class SettingsRepositorySharedPrefsTest {
    private lateinit var ctx: Context
    private lateinit var repo: SettingsRepository

    @Before fun setUp() {
        ctx = ApplicationProvider.getApplicationContext()
        repo = SettingsRepository(ctx.getSharedPreferences("test", Context.MODE_PRIVATE))
    }

    @Test fun `theme persists across reads`() {
        repo.setTheme("dark")
        assertEquals("dark", repo.theme())
    }
}
```

DataStore (preferred)

```
class SettingsDataStoreTest {
    @TempDir lateinit var tempDir: File
    private lateinit var dataStore: DataStore<Preferences>
    private lateinit var repo: SettingsRepository

    @BeforeEach fun setUp() {
        dataStore = PreferenceDataStoreFactory.create(
            scope = TestScope(),
            produceFile = { File(tempDir, "settings.preferences_pb") }
        )
        repo = SettingsRepository(dataStore)
    }

    @Test fun `theme write then read`() = runTest {
        repo.setTheme("dark")
        assertEquals("dark", repo.themeFlow().first())
    }
}
```

Key trick: create the DataStore with a **temporary directory** so each test has fresh state and there's no file conflict between tests running in parallel.

17. Test with Hilt — full integration tests

For integration tests where you want the real DI graph (e.g., "does my app actually wire together?"), use Hilt's testing artifacts.

```
// 1. Replace the production module with a test module
@TestInstallIn(components = [SingletonComponent::class], replaces = [DataModule::class])
@Module
object FakeDataModule {
    @Provides @Singleton
    fun provideRepo(): NotesRepository = FakeNotesRepository()
}

// 2. Use a Hilt-aware test runner
@CustomTestApplication(BaseApp::class)
interface HiltTestApplication               // Generates HiltTestApplication_Application

class HiltTestRunner : AndroidJUnitRunner() {
    override fun newApplication(cl: ClassLoader?, name: String?, ctx: Context?): Application {
        return super.newApplication(cl, HiltTestApplication_Application::class.java.name, ctx)
    }
}
// build.gradle.kts: testInstrumentationRunner = "com.example.HiltTestRunner"

// 3. Write the test
@HiltAndroidTest
@RunWith(AndroidJUnit4::class)
class NotesScreenIntegrationTest {
    @get:Rule(order = 0) val hilt = HiltAndroidRule(this)
    @get:Rule(order = 1) val compose = createAndroidComposeRule<HiltTestActivity>()

    @Inject lateinit var repo: NotesRepository                 // auto-injected = FakeNotesRepository

    @Before fun init() { hilt.inject() }

    @Test fun `screen shows notes from repo`() {
        runBlocking { (repo as FakeNotesRepository).seed(listOf(Note("1", "Hello", "", 0))) }
        compose.setContent { NotesScreen() }
        compose.onNodeWithText("Hello").assertIsDisplayed()
    }
}
```

18. KMP-friendly testing — write tests in `commonTest`

For Kotlin Multiplatform modules, prefer test code in `commonTest` so the same tests run on Android, iOS, JVM, and JS.

FEATURE	KMP-FRIENDLY CHOICE	AVOID
Test framework	Kotest (`FunSpec`, `BehaviorSpec`) or `kotlin.test`	JUnit 5 in commonTest (JVM-only)
Mocking	Hand-written fakes / Mockative	MockK (JVM-only without extras)
Coroutines	`kotlinx-coroutines-test` (KMP)	—
Flow assertion	Turbine (KMP since 1.0)	—
Network mocking	Ktor MockEngine	MockWebServer (JVM-only)
DB	SQLDelight test driver	Room (Android-only until 2.7)
Time	`Clock` interface, inject in tests	`System.currentTimeMillis`

```kotlin
// commonTest/kotlin/UserUseCaseTest.kt — runs on Android, iOS, JVM, JS
class UserUseCaseTest : FunSpec({
    test("returns user when id valid") {
        val repo = FakeUserRepository().apply { seed(User("1", "Sai")) }
        val useCase = GetUserUseCase(repo)
        val result = useCase("1")
        result.shouldBe(User("1", "Sai"))
    }
})
```

19. Coroutines & Flow testing — deep dive

The 3 helpers you'll use everywhere

HELPER	WHAT IT DOES
`runTest { }`	Run suspending test code with virtual time. Replaces `runBlocking` in tests.
`StandardTestDispatcher`	Doesn't auto-run coroutines; you call `runCurrent()` / `advanceTimeBy(ms)` / `advanceUntilIdle()`. Use when timing matters.
`UnconfinedTestDispatcher`	Auto-runs each coroutine eagerly until first suspension. Easier but less precise.

```
@Test fun `debounce + flatMapLatest cancels prior request`() = runTest {
    val queries = MutableSharedFlow<String>(replay = 0)
    val calls = mutableListOf<String>()

    val job = launch {
        queries
            .debounce(300)
            .flatMapLatest { q -> flow { calls += q; emit("result $q") } }
            .collect()
    }

    queries.emit("a")
    advanceTimeBy(100)                    // not yet past debounce
    queries.emit("ab")                    // resets debounce timer
    advanceTimeBy(300)                    // past debounce since last emit
    runCurrent()

    assertEquals(listOf("ab"), calls)     // only the latest reached the producer
    job.cancel()
}
```

Turbine for Flow assertions

```
flow.test {
    assertEquals(1, awaitItem())
    assertEquals(2, awaitItem())
    awaitComplete()                           // or cancelAndIgnoreRemainingEvents()
}

// Wait with timeout
flow.test(timeout = 2.seconds) {
    expectMostRecentItem()                    // drains buffered items, returns latest
}
```

20. Screenshot testing — Paparazzi (View) and Roborazzi (Compose)

Render UI to PNG without an emulator and diff against a "golden" image. Catches design regressions instantly. Cheap to run on CI.

```
// Paparazzi (Compose-friendly via the snapshot lambda)
class NotesScreenScreenshotTest {
    @get:Rule val paparazzi = Paparazzi(deviceConfig = PIXEL_5)

    @Test fun notes_empty() {
        paparazzi.snapshot {
            NotesScreen(state = NotesScreenState(notes = emptyList()), onAdd = {})
        }
    }

    @Test fun notes_populated() {
        paparazzi.snapshot {
            NotesScreen(
                state = NotesScreenState(notes = sampleNotes()),
                onAdd = {}
            )
        }
    }

    @Test fun notes_dark_theme() {
        paparazzi.snapshot {
            AppTheme(darkTheme = true) {
                NotesScreen(state = NotesScreenState(notes = sampleNotes()), onAdd = {})
            }
        }
    }
}
```

Workflow: first run captures golden images (commit them to git). Future CI runs re-render and diff. Diffs show as a generated HTML report.

Roborazzi uses Robolectric so it's fully Compose-aware (some Compose features that Paparazzi struggles with work in Roborazzi). Pick whichever your team prefers — both run on JVM only.

21. End-to-end testing — Maestro & UI Automator

For full-flow tests (login → checkout → confirmation), Compose tests are too tightly coupled to one screen. Use:

TOOL	WHAT IT DOES
Maestro	YAML-based E2E flow automation. Far simpler than Espresso for E2E. Hot-reload, video recordings.
UI Automator	Cross-app testing (e.g., your app + system settings). Lower-level than Maestro.
Espresso	Use for mid-complexity in-app flows when you don't want Maestro setup. Compose Test now covers most use cases.

```yaml
# sample-flow.yaml (Maestro)
appId: com.example.notes
---
- launchApp
- tapOn: "Add"
- inputText: "My note"
- tapOn: "Save"
- assertVisible: "My note"
```

Run with `maestro test sample-flow.yaml`. Add to CI for critical flows only — keep the count tiny.

22. Macrobenchmark — measuring performance

For perf regressions (cold start, scroll jank, key-flow timing), use Macrobenchmark — runs against a release build on a real device.

```fun
@RunWith(AndroidJUnit4::class)
class StartupBenchmark {
    @get:Rule val rule = MacrobenchmarkRule()

    @Test fun coldStartup() = rule.measureRepeated(
        packageName = "com.example.notes",
        metrics = listOf(StartupTimingMetric()),
        iterations = 5,
        startupMode = StartupMode.COLD
    ) {
        pressHome()
        startActivityAndWait()
    }

    @Test fun scrollPerf() = rule.measureRepeated(
        packageName = "com.example.notes",
        metrics = listOf(FrameTimingMetric()),
        iterations = 5,
        startupMode = StartupMode.WARM
    ) {
        pressHome(); startActivityAndWait()
        device.findObject(By.res("feed")).fling(Direction.DOWN)
    }
}
```

Pair with **Baseline Profile generation** (`BaselineProfileRule`) — same Macrobenchmark machinery, but records hot methods to ship in your APK for AOT compilation.

23. Code coverage — Kover (KMP) & JaCoCo (JVM)

```
// build.gradle.kts
plugins {
    id("org.jetbrains.kotlinx.kover") version "0.7.6"
}

kover {
    reports {
        verify {
            rule {
                bound { minValue = 80 }    // fail CI if < 80% line coverage
            }
        }
    }
}
```

Run: `./gradlew koverHtmlReport koverVerify`.

> **Coverage trap:** 100% coverage doesn't mean 100% tested. A test that calls a function and asserts nothing scores full coverage and tests nothing. Use coverage as a floor, not a goal.

24. Common mistakes & anti-patterns

DON'T

- Test private methods directly (test through public)
- Mock everything (over-coupled to implementation)
- Use `Thread.sleep` in coroutine tests (use virtual time)
- Share mutable fixtures across tests (flaky test cascade)
- Test framework code (Room generated class is Google's responsibility)
- Write tests that depend on test order (each test must be independent)
- Use `runBlocking` in tests (use `runTest`)
- Hit real network in unit tests (use MockWebServer)
- Skip writing tests because "I'll add them later" (you won't)
- Pursue 100% coverage (diminishing returns; flaky-test risk)
- Mock data classes (just construct them with values)
- Mock final classes when a fake works (Kotlin classes are final by default — fakes are usually cleaner)

DO

- Hoist state out of Composables — makes UI tests trivial
- Inject dispatchers and clock
- Use fakes for repositories, mocks only for verification
- Run tests on every PR via CI
- Treat flaky tests as bugs — fix or delete, never ignore
- Write the test before the bug fix (proves the fix works)
- Use `@TempDir` for any file-based test fixtures
- Name tests as full sentences — they're documentation
- Group related tests in nested classes (JUnit 5 `@Nested`)
- Use parameterized tests for "same logic, many inputs"
- Profile slow tests; aim for unit tests under 100 ms each

25. Interview Q&A — drill answers

Q: "Fakes vs mocks — which do you prefer and why?"

A: Fakes for almost everything that has substantial behavior (Repository, DataStore, Api). Mocks only when verifying interaction is the actual property under test (analytics, logger). Reasons: fakes survive refactors; tests look like normal code; hard-to-test contracts get caught (your fake exhibits the same edge cases). Mocks tightly couple tests to implementation; refactor breaks 50 mock setups.

Q: "How do you test a ViewModel that uses Coroutines and Flow?"

A: Three pieces. (1) `Dispatchers.setMain(StandardTestDispatcher())` in setUp + `resetMain()` in tearDown — viewModelScope uses Dispatchers.Main which is unavailable in JVM tests by default. (2) Wrap the test body in

`runTest(dispatcher)` for virtual time + `advanceUntilIdle()` to drain. (3) Use Turbine (`flow.test { awaitItem() }`) for asserting Flow emissions sequentially.

Q: "How do you test a Repository?"

A: Inject fake DAO and fake Api into the real Repository implementation. Verify the data orchestration logic (e.g., "sync writes API results to DAO", "save persists locally and queues remote"). For DAO/Api themselves, separate tests — DAO with in-memory Room (instrumented), Api with MockWebServer.

Q: "How do you test a Composable?"

A: Hoist state out — make it a stateless Composable taking `(state, callbacks)`. Then `createComposeRule()`, `setContent { MyScreen(state, callback) }`, `onNodeWithText/Tag` for selectors, `performClick` / `performTextInput` for interactions, `assertIsDisplayed` / `assertTextEquals` for assertions. The full screen with ViewModel just calls into the stateless one — separate Hilt integration test for the wiring.

Q: "When would you use Robolectric vs instrumented tests?"

A: Robolectric runs Android API on JVM (fast, no emulator). Use for: SharedPreferences/Service/legacy Android API tests where you want speed. Instrumented tests run on a real device/emulator (slow but accurate). Use for: Room DAO (in-memory database), Compose UI tests, anything that needs real GL / hardware acceleration / system services beyond Robolectric's shadows.

Q: "What's the testing pyramid and why does shape matter?"

A: Many fast cheap unit tests at the base, fewer integration tests in the middle, very few slow expensive UI tests at the top. Shape matters because UI tests are 10× slower and 10× flakier than unit tests for the same coverage. Inverted pyramids (many UI tests, few unit tests) make CI slow, flaky, and expensive — bug detection happens late on the cost curve.

Q: "How do you write testable code?"

A: Five things: (1) constructor injection, never `val x = SomeStatic.create()`; (2) depend on interfaces, not implementations; (3) inject dispatcher and clock; (4) keep functions pure where possible; (5) hoist state in Compose. The pattern: every external dependency is a parameter; every parameter can be a fake.

Q: "Coroutine test — why `runTest` and not `runBlocking`?"

A: `runTest` uses virtual time — `delay(60_000)` resolves immediately. Tests stay fast. `runBlocking` blocks real threads, makes tests slow and timing-sensitive. Use `runTest` for any test that touches coroutines, `runBlocking` only for bridging to legacy non-suspend code in test setup.

Q: "Why prefer MockK over Mockito for Kotlin?"

A: MockK supports Kotlin features Mockito struggles with: final classes by default (Kotlin classes are final unless marked `open`), suspend functions (`coEvery` / `coVerify`), top-level functions, object/companion mocking, lambda captures. With Mockito you need `mockito-inline` for finals + manual setup for coroutines.

Q: "How do you test Hilt?"

A: Annotate test class `@HiltAndroidTest`, add `HiltAndroidRule`, call `hilt.inject()` in `@Before`. Replace production modules in tests via `@TestInstallIn(replaces = [DataModule::class])` on a fake module. Use a

custom Application for instrumented tests — generate via `@CustomTestApplication`.

Q: "How do you test code that depends on time?"

A: Inject a `Clock` interface (`fun now(): Long`). Production: real Clock returning `System.currentTimeMillis()`. Tests: `FixedClock(100L)` returning a known value. Same trick for any system call — wrap behind an interface, inject, override in tests.

Q: "Should we test private methods?"

A: No. Test through public methods; if a private method is too complex to be exercised that way, extract it to its own class with public methods. Testing privates couples tests to implementation; you'll be unable to refactor without rewriting tests.

Q: "What's a flaky test? How do you handle it?"

A: A test that sometimes passes, sometimes fails with no code change. Causes: timing dependencies, shared state between tests, race conditions, network. Treat flaky tests as bugs — fix or delete, never ignore. Common fixes: use virtual time (`runTest`), isolate fixtures (`@TempDir`), use stable selectors (`testTag`), eliminate `Thread.sleep`.

26. One-page recall card

The mental model

UI = f(state). Test logic in isolation (unit), test wiring across layers (integration), test critical user journeys (E2E). Push every test as low on the pyramid as possible. Design for testability: inject everything, depend on interfaces.

The latest stack (2026)

- JUnit 5 (JVM) / Kotest (KMP)
- MockK (mocks) + hand-written fakes (preferred for repos)
- `kotlinx-coroutines-test` + Turbine
- MockWebServer for HTTP
- Room in-memory for DAO
- Robolectric for Android API on JVM
- Compose Test rule for UI
- Hilt testing for DI integration
- Paparazzi / Roborazzi for screenshots
- Macrobenchmark for perf
- Maestro for E2E flows
- Kover / JaCoCo for coverage

Layer → testing approach

- Plain class → JUnit 5
- UseCase → JUnit 5 + fakes
- Repository → JUnit 5 + fake DAO + fake Api
- ViewModel → JUnit 5 + setMain dispatcher + Turbine
- Room DAO → instrumented + in-memory DB
- Retrofit Api → JUnit 5 + MockWebServer
- Composable → Compose test rule (hoist state for testability)
- Service → extract logic to plain class, test that
- ContentProvider → wrap in DataSource interface, test through that

- SharedPrefs / DataStore → temp directory + real instance

Testability principles

- Constructor injection
- Depend on interfaces
- Inject dispatcher + clock
- Pure functions
- Hoist Compose state
- No hidden global state

Avoid

- `runBlocking` in tests
- Mocks for repositories (use fakes)
- Testing private methods
- Shared mutable fixtures
- `Thread.sleep`
- Real network in unit tests
- Pursuing 100% coverage
- Order-dependent tests
- Flaky tests left in CI

Read top-to-bottom for understanding · skim §24 (mistakes), §25 (Q&A), §26 (recall) before any interview. Pair with the **Architecture Patterns**, **Coroutines**, and **Compose** cheat sheets in this repo.